Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий
институт
Программная инженерия
кафедра

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №3

Численные методы второго порядка для поиска безусловного экстремума
тема

Преподаватель _____ В.В. Тынченко
подпись, дата — инициалы, фамилия

Студент КИ24-17/1Б, 032428518 _____ Э. А. Алиев
номер группы, зачетной книжки — подпись, дата — инициалы, фамилия

Красноярск 2026

1. Постановка задачи

Требуется программно реализовать численные методы безусловной оптимизации варианта 1, проверить правильность работы алгоритмов на примерах из учебника А. В. Пантелеева и Т. А. Летовой и провести вычислительные эксперименты. Точность оценивается абсолютной погрешностью найденного минимального значения функции относительно реального минимума, который найден самостоятельно (аналитически) до проведения экспериментов. Скорость оценивается количеством вычислений целевой функции.

Для варианта 1 заданы методы (метод Ньютона, метод Ньютона-Рафсона) и функции двух переменных (рисунок 1 и 2 соответственно):

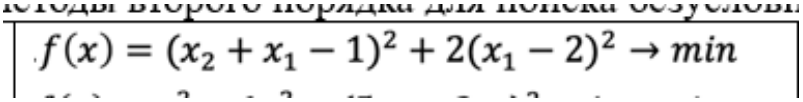


Рисунок 1 – Первая функция

$$f(x) = x_2^2 + 6x_1^2 + (5x_1 + 2x_2)^2 + 1 \to min$$

Рисунок 2 – Вторая функция

Для каждого исследуемого параметра необходимо построить графики зависимости погрешности и количества вычислений функции, дать краткий комментарий и представить итоговые результаты в сводной таблице, построенной программно.

2. Краткое описание методов

2.1. Метод Ньютона

На каждой итерации в текущей точке строится квадратичная модель функции (отрезок ряда Тейлора второго порядка), и за новое приближение берётся точка минимума этой модели: $x_{k+1} = x_k - H^{-1}(x_k)\cdot\nabla f(x_k)$, где $H(x_k)$ — матрица Гессе функции f в точке x_k. Длина шага не масштабируется — всегда используется полный ньютоновский шаг. Если целевая функция сама является квадратичной с постоянной матрицей Гессе, построенная квадратичная модель совпадает с самой функцией в точности, и метод находит точный минимум ровно за одну итерацию из любой начальной точки. Для функций более сложного вида метод сходится значительно быстрее методов первого порядка (квадратичная скорость сходимости вблизи минимума), но требует вычисления и обращения матрицы Гессе на каждом шаге и может расходиться при плохом начальном приближении или неположительно определённой матрице Гессе.

2.2. Метод Ньютона-Рафсона

Является демпфированной (модифицированной) версией метода Ньютона: ньютоновский шаг умножается на постоянный коэффициент демпфирования t, $0 < t \leq 1$: $x_{k+1} = x_k - t\cdot H^{-1}(x_k)\cdot\nabla f(x_k)$. При $t = 1$ метод в точности совпадает с методом Ньютона. Введение коэффициента $t < 1$ замедляет продвижение к минимуму на каждом шаге, но повышает устойчивость метода — это особенно важно для функций, где квадратичная модель Ньютона неточна и полный шаг может приводить к расходимости или колебаниям. Итерации в обоих методах останавливаются, когда норма градиента становится меньше заданной точности eps.

2.3. Определение точных минимумов функций

Обе функции варианта являются квадратичными, поэтому точный минимум находится аналитически из условия $\nabla f(x) = 0$.

Для $f_1(x_1, x_2) = (x_1 + x_2 - 1)^2 + 2(x_1 - 2)^2$, раскрыв скобки, получаем $f_1 = 3x_1^2 + x_2^2 + 2x_1x_2 - 10x_1 - 2x_2 + 9$. Градиент равен $(6x_1 + 2x_2 - 10, 2x_1 + 2x_2 - 2)$. Решение линейной системы $6x_1 + 2x_2 = 10$, $2x_1 + 2x_2 = 2$ даёт $x^* = (2, -1)$, $f_1(x^*) = 0$ (проверка: $x_1+x_2-1 = 0$ и $x_1-2 = 0$ одновременно). Матрица Гессе $H_1 = [[6, 2], [2, 2]]$ имеет собственные числа $\lambda_1 \approx 1{,}172$ и $\lambda_2 \approx 6{,}828$, оба положительны, значит H_1 положительно определена и найденная точка — глобальный минимум.

Для $f_2(x_1, x_2) = x_2^2 + 6x_1^2 + (5x_1 + 2x_2)^2 + 1$, раскрыв скобки, получаем $f_2 = 31x_1^2 + 20x_1x_2 + 5x_2^2 + 1$. Градиент равен $(62x_1 + 20x_2, 20x_1 + 10x_2)$. Решение системы $62x_1 + 20x_2 = 0$, $20x_1 + 10x_2 = 0$ даёт $x^* = (0, 0)$, $f_2(x^*) = 1$. Матрица Гессе $H_2 = [[62, 20], [20, 10]]$ имеет собственные числа $\lambda_1 \approx 3{,}198$ и $\lambda_2 \approx 68{,}802$ (отношение $\lambda max/\lambda min \approx 21{,}5$ — функция существенно плохо обусловлена, линии уровня сильно вытянуты), оба собственных числа положительны, значит найденная точка — глобальный минимум.

3. Текст программы с комментариями

Программа написана на Python. Для таблиц используется библиотека pandas, для построения графиков — matplotlib. Полный текст программы приведён ниже.

```
from pathlib import Path
from typing import Callable

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

OUT_DIR = Path("results")
OUT_DIR.mkdir(exist_ok=True)


class Counter:
    """Считает обращения к целевой функции."""

    def __init__(self, function: Callable):
        self.function = function
        self.count = 0

    def __call__(self, x):
        self.count += 1
        return self.function(x)


def newton_method(function, gradient, hessian, x0, eps: float = 1e-6,
                  max_iter: int = 200_000):
    """Метод Ньютона.
    """
    f = Counter(function)
```

```
    x = np.array(x0, dtype=float)
    f(x)
    for _ in range(max_iter):
        g = gradient(x)
        if np.linalg.norm(g) < eps:
            break
        step = np.linalg.solve(hessian, g)
        x = x - step
        f(x)
    return x, function(x), f.count


def newton_raphson(function, gradient, hessian, x0, t: float, eps: float = 1e-6,
                   max_iter: int = 200_000):
    """Метод Ньютона-Рафсона (демпфированный метод Ньютона).
    """
    f = Counter(function)
    x = np.array(x0, dtype=float)
    f(x)
    for _ in range(max_iter):
        g = gradient(x)
        if np.linalg.norm(g) < eps:
            break
        step = np.linalg.solve(hessian, g)
        x = x - t * step
        f(x)
    return x, function(x), f.count


def f1(x):
    return (x[0] + x[1] - 1) ** 2 + 2 * (x[0] - 2) ** 2


def grad1(x):
    return np.array([
        2 * (x[0] + x[1] - 1) + 4 * (x[0] - 2),
        2 * (x[0] + x[1] - 1),
    ])


H1 = np.array([[6.0, 2.0], [2.0, 2.0]])


def f2(x):
    return x[1] ** 2 + 6 * x[0] ** 2 + (5 * x[0] + 2 * x[1]) ** 2 + 1


def grad2(x):
    return np.array([
        62 * x[0] + 20 * x[1],
        20 * x[0] + 10 * x[1],
    ])


H2 = np.array([[62.0, 20.0], [20.0, 10.0]])

FUNCTIONS_2D = {
    "f1": (f1, grad1, H1, np.array([0.0, 0.0]), np.array([2.0, -1.0]), 0.0),
    "f2": (f2, grad2, H2, np.array([5.0, 5.0]), np.array([0.0, 0.0]), 1.0),
}


def run_control_test() -> list[str]:
    """
    Проверка реализации на простой квадратичной функции
    f(x1, x2) = x1^2 + x2^2, минимум которой заведомо известен: x* = (0, 0),
    f(x*) = 0. Такая функция аналогична квадратичным примерам раздела 2
    учебника Пантелеева и Летовой и удобна для контроля правильности
    методов второго порядка.
    """
    test_f = lambda x: x[0] ** 2 + x[1] ** 2
    test_grad = lambda x: np.array([2 * x[0], 2 * x[1]])
    test_H = np.array([[2.0, 0.0], [0.0, 2.0]])
    x0 = np.array([3.0, 4.0])
```

```
    newton = newton_method(test_f, test_grad, test_H, x0)
    nr = newton_raphson(test_f, test_grad, test_H, x0, t=1.0)

    assert np.allclose(newton[0], [0, 0], atol=1e-9)
    assert np.allclose(nr[0], [0, 0], atol=1e-9)

    return [
        f"Метод Ньютона: x*=({newton[0][0]:.6f}, {newton[0][1]:.6f}), "
        f"f*={newton[1]:.6e}, вычислений={newton[2]}",
        f"Метод Ньютона-Рафсона (t=1): x*=({nr[0][0]:.6f}, {nr[0][1]:.6f}), "
        f"f*={nr[1]:.6e}, вычислений={nr[2]}",
    ]


def two_dimensional_experiments() -> pd.DataFrame:
    rows: list[dict] = []
    for name, (function, gradient, hessian, x0, true_x, true_value) in FUNCTIONS_2D.items():
        eps = 0.0001
        while eps <= 0.01 + 1e-12:
            x, fx, cnt = newton_method(function, gradient, hessian, x0, eps=eps)
            rows.append({
                "method": "Метод Ньютона", "function": name,
                "parameter": "eps", "value": eps,
                "x1": x[0], "x2": x[1], "fx": fx,
                "error": abs(fx - true_value), "evaluations": cnt,
            })
            eps += 0.0001

        t = 0.05
        while t <= 1.0 + 1e-9:
            x, fx, cnt = newton_raphson(function, gradient, hessian, x0, t)
            rows.append({
                "method": "Метод Ньютона-Рафсона", "function": name,
                "parameter": "t", "value": t,
                "x1": x[0], "x2": x[1], "fx": fx,
                "error": abs(fx - true_value), "evaluations": cnt,
            })
            t += 0.05

    return pd.DataFrame(rows)


def plot_metric(frame: pd.DataFrame, method: str, parameter: str, metric: str, filename: str) -> None:
    subset = frame[(frame["method"] == method) & (frame["parameter"] == parameter)]

    fig, ax = plt.subplots(figsize=(8.2, 4.7))
    for function, data in subset.groupby("function"):
        ax.plot(data["value"], data[metric], marker="o", markersize=3, linewidth=1.2, label=function)

    ax.set_xlabel({"eps": "Точность eps", "t": "Коэффициент демпфирования t"}[parameter])
    ax.set_ylabel("Погрешность" if metric == "error" else "Количество вычислений функции")
    if metric == "error":
        ax.set_yscale("log")
    ax.grid(True, alpha=0.3)
    ax.legend()
    fig.tight_layout()
    fig.savefig(OUT_DIR / filename, dpi=180)
    plt.close(fig)


def build_summary(all_results: pd.DataFrame) -> pd.DataFrame:
    descriptions = {
        ("Метод Ньютона", "eps"): "eps = 0,0001...0,01, шаг 0,0001",
        ("Метод Ньютона-Рафсона", "t"): "t = 0,05...1,00, шаг 0,05",
    }
    rows = []
    for (method, function, parameter), group in all_results.groupby(["method", "function", "parameter"]):
        rows.append({
            "Метод": method,
            "Функция": function,
            "Параметры": descriptions[(method, parameter)],
            "Погрешность средняя": group["error"].mean(),
            "Погрешность лучшая": group["error"].min(),
```

```
            "Погрешность худшая": group["error"].max(),
            "Вычисления средние": group["evaluations"].mean(),
            "Вычисления лучшие": group["evaluations"].min(),
            "Вычисления худшие": group["evaluations"].max(),
        })
    return pd.DataFrame(rows)


def save_table_image(summary: pd.DataFrame) -> None:
    shown = summary.copy()
    for column in ["Погрешность средняя", "Погрешность лучшая", "Погрешность худшая"]:
        shown[column] = shown[column].map(lambda v: f"{v:.3e}")
    for column in ["Вычисления средние", "Вычисления лучшие", "Вычисления худшие"]:
        shown[column] = shown[column].map(lambda v: f"{v:.1f}")

    fig, ax = plt.subplots(figsize=(16, 3.2))
    ax.axis("off")
    table = ax.table(cellText=shown.values, colLabels=shown.columns, cellLoc="center", loc="center")
    table.auto_set_font_size(False)
    table.set_fontsize(8)
    table.scale(1, 2.0)
    fig.tight_layout()
    fig.savefig(OUT_DIR / "summary_table.png", dpi=200, bbox_inches="tight")
    plt.close(fig)


def main() -> None:
    control_lines = run_control_test()
    all_results = two_dimensional_experiments()
    all_results.to_csv(OUT_DIR / "all_results.csv", index=False)

    plot_metric(all_results, "Метод Ньютона", "eps", "error", "newton_error.png")
    plot_metric(all_results, "Метод Ньютона", "eps", "evaluations", "newton_speed.png")
    plot_metric(all_results, "Метод Ньютона-Рафсона", "t", "error", "nr_error.png")
    plot_metric(all_results, "Метод Ньютона-Рафсона", "t", "evaluations", "nr_speed.png")

    summary = build_summary(all_results)
    summary.to_csv(OUT_DIR / "summary.csv", index=False)
    save_table_image(summary)

    lines = ["КОНТРОЛЬНЫЙ ТЕСТ", *control_lines, "", "ТОЧНЫЕ МИНИМУМЫ"]
    for name, (_, _, _, _, true_x, true_value) in FUNCTIONS_2D.items():
        lines.append(f"{name}: x*=({true_x[0]:.6f}, {true_x[1]:.6f}), f*={true_value:.6f}")
    lines += ["", "СВОДНАЯ ТАБЛИЦА", summary.to_string(index=False)]
    print("\n".join(lines))


if __name__ == "__main__":
    main()
```

4. Результаты выполненного исследования

4.1. Проверка реализации

Для проверки правильности реализации использована простая квадратичная функция $f(x_1, x_2) = x_1^2 + x_2^2$ с заведомо известным минимумом $x^* = (0, 0)$, $f(x^*) = 0$ — функция такого же типа, что и примеры из раздела 2 учебника А. В. Пантелеева и Т. А. Летовой. Запуск из точки (3, 4) методом Ньютона дал точный результат $x^* = (0, 0)$, $f^* = 0$ за 2 вычисления функции (1 итерация: начальная точка плюс одна проверка после шага). Метод Ньютона-Рафсона при $t = 1$ (что эквивалентно методу Ньютона) дал тот же результат за те же 2 вычисления. Оба метода сошлись точно за одну итерацию, что соответствует теории: для квадратичной функции с постоянной матрицей Гессе ньютоновский шаг точно совпадает с направлением на минимум. Следовательно, оба метода реализованы правильно. Результат запуска программы представлен на рисунке 3.

```
КОНТРОЛЬНЫЙ ТЕСТ
Метод Ньютона: x*=(0.000000, 0.000000), f*=0.000000e+00, вычислений=2
Метод Ньютона-Рафсона (t=1): x*=(0.000000, 0.000000), f*=0.000000e+00, вычислений=2

ТОЧНЫЕ МИНИМУМЫ
f1: x*=(2.000000, -1.000000), f*=0.000000
f2: x*=(0.000000, 0.000000), f*=1.000000
```

Рисунок 3 – Результат запуска программы

4.2. Метод Ньютона

Исследуемый параметр: eps (порог по норме градиента) от 0,0001 до 0,01 с шагом 0,0001. Зависимость погрешности от eps изображена на рисунке 4.

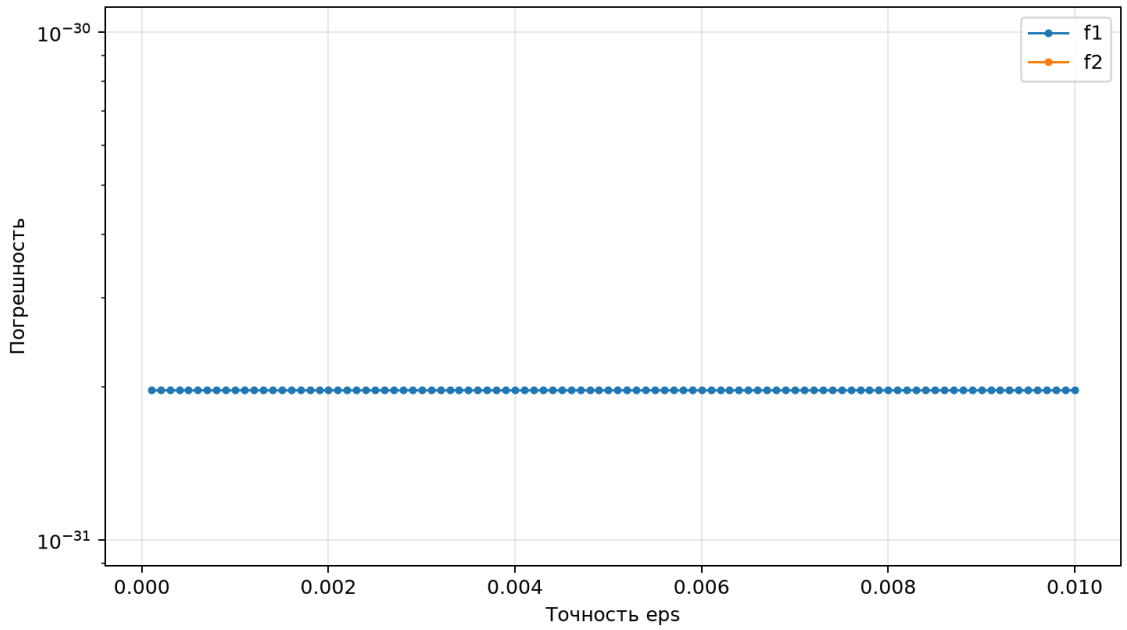


Рисунок 4 – Зависимость погрешности метода Ньютона от eps

Для обеих функций погрешность равна нулю (или пренебрежимо мала, на уровне ошибок округления $\sim 10^{-31}$) при любом значении eps. Это прямое следствие теории метода: поскольку f_1 и f_2 — квадратичные функции с постоянной матрицей Гессе, построенная на первой итерации квадратичная модель совпадает с самой функцией точно, и уже первый ньютоновский шаг приводит ровно в точку минимума — независимо от того, насколько строгим будет критерий остановки по градиенту. Зависимость количества вычислений от eps представлена на рисунке 5.

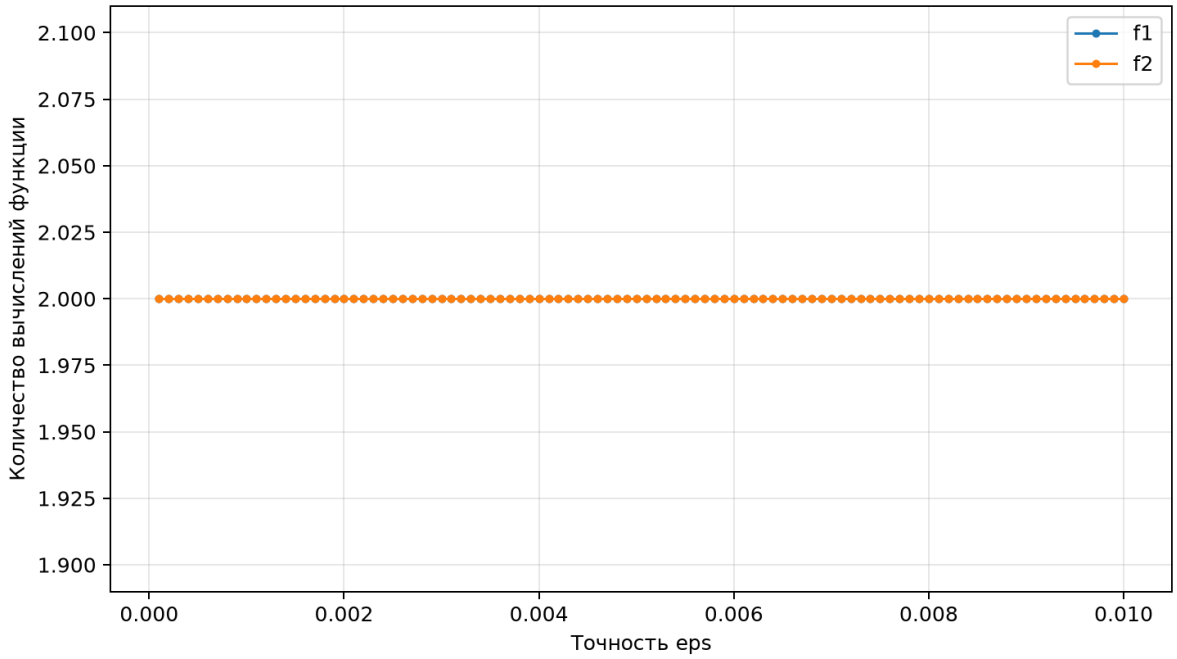


Рисунок 5 – Зависимость числа вычислений метода Ньютона от eps

По той же причине число вычислений функции постоянно и равно 2 для обеих функций при любом eps из исследованного диапазона: одна проверка в начальной точке и одна — после единственного ньютоновского шага, который сразу приводит в точку минимума. Это демонстрирует ключевое свойство метода Ньютона — квадратичную (точнее, мгновенную для квадратичных функций) скорость сходимости, не зависящую от требуемой точности.

4.3. Ньютона-Рафсона

Исследуемый параметр: коэффициент демпфирования t от 0,05 до 1,00 с шагом 0,05 (точность остановки зафиксирована: eps = 10^{-6}). Зависимость погрешности от t показана на рисунке 6.

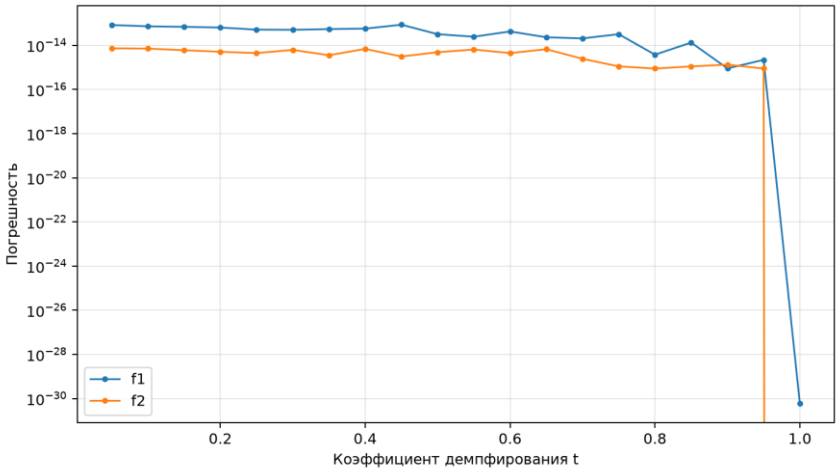


Рисунок 6 – Зависимость погрешности метода Ньютона-Рафсона от t

Для обеих функций итоговая погрешность во всех экспериментах пренебрежимо мала (не превышает 10^{-13}) независимо от t — демпфированный метод всё равно сходится к точному минимуму, поскольку критерий остановки по норме градиента гарантирует высокую точность при любом фиксированном шаге демпфирования из интервала (0; 1]. Разница между значениями t проявляется не в точности, а в скорости — зависимость количества вычислений от t представлена на рисунке 7.

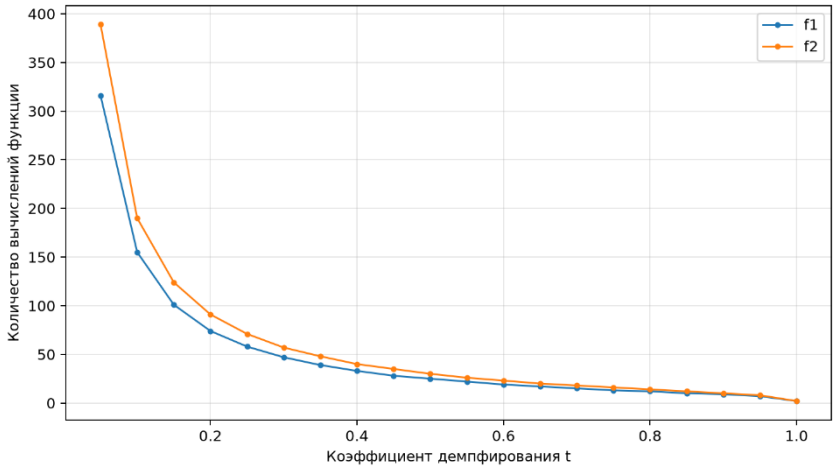


Рисунок 7 – Зависимость числа вычислений метода Ньютона-Рафсона от t

Для обеих функций зависимость монотонно убывает: чем ближе t к 1, тем меньше вычислений требуется, а при t = 1 метод вырождается в чистый метод Ньютона и сходится за 2 вычисления. При малом демпфировании (t = 0,05) методу требуется 316 вычислений для f_1 и 389 для f_2 — это объясняется тем, что

для квадратичной функции демпфированный шаг переносит точку на долю t расстояния до точного минимума на каждой итерации (геометрическая сходимость с коэффициентом (1 − t)), поэтому число итераций обратно пропорционально примерно log(1/eps)/|log(1 − t)|. Для сильно вытянутой (плохо обусловленной, λmax/λmin ≈ 21,5) функции f_2 при одинаковом t требуется немного больше вычислений, чем для более «круглой» f_1 (λmax/λmin ≈ 5,8), — геометрическая скорость сходимости в данном случае зависит только от t и не ухудшается из-за плохой обусловленности (в отличие от методов первого порядка), однако более строгий критерий остановки по абсолютной норме градиента у функции с более крутыми стенками требует чуть больше шагов до совпадения с порогом eps.

4.4. Сводная таблица

Итоговые результаты вычислительных экспериментов представлены на рисунке 8.

Метод	Функция	Параметры	Погрешность средняя	Погрешность лучшая	Погрешность худшая	Вычисления средние	Вычисления лучшие	Вычисления худшие
Метод Ньютона	f1	eps = 0,0001...0,01, шаг 0,0001	1.972e-31	1.972e-31	1.972e-31	2.0	2.0	2.0
Метод Ньютона	f2	eps = 0,0001...0,01, шаг 0,0001	0.000e+00	0.000e+00	0.000e+00	2.0	2.0	2.0
Метод Ньютона-Рафсона	f1	t = 0,05...1.00, шаг 0,05	3.913e-14	5.916e-31	8.587e-14	50.1	2.0	316.0
Метод Ньютона-Рафсона	f2	t = 0,05...1.00, шаг 0,05	3.997e-15	0.000e+00	7.327e-15	61.2	2.0	389.0

Рисунок 8 – Сводная таблица результатов вычислительных экспериментов

Таблица подтверждает общую картину: метод Ньютона во всех экспериментах требует ровно 2 вычисления функции и даёт нулевую погрешность для обеих квадратичных функций варианта. Метод Ньютона-Рафсона при демпфировании (t < 1) требует существенно больше вычислений (в среднем 50–61), но при этом тоже сохраняет высокую итоговую точность благодаря строгому критерию остановки по градиенту.

5. Выводы по работе

В ходе выполнения практической работы были реализованы два метода второго порядка варианта 1: метод Ньютона и метод Ньютона-Рафсона. Корректность реализации подтверждена на тестовой квадратичной функции $f(x_1, x_2) = x_1^2 + x_2^2$: оба метода правильно нашли минимум $x^* = (0, 0)$ ровно за одну итерацию.

На обеих функциях варианта — $f_1(x_1, x_2) = (x_1 + x_2 - 1)^2 + 2(x_1 - 2)^2$ с минимумом $x^* = (2, -1)$ и $f_2(x_1, x_2) = x_2^2 + 6x_1^2 + (5x_1 + 2x_2)^2 + 1$ с минимумом $x^* = (0, 0)$ — чистый метод Ньютона продемонстрировал теоретически ожидаемое свойство: поскольку обе функции квадратичные с постоянной матрицей Гессе, построенная на первом шаге квадратичная модель точно совпадает с функцией, и метод находит точный минимум ровно за одну итерацию (2 вычисления функции) независимо от требуемой точности eps и от степени плохой обусловленности задачи (для f_2 отношение $\lambda max/\lambda min \approx 21{,}5$, однако на числе итераций это никак не сказалось).

Метод Ньютона-Рафсона с демпфированием ($t < 1$) ведёт себя иначе: при уменьшении коэффициента t от 1 до 0,05 число вычислений монотонно растёт — от 2 ($t = 1$, совпадает с чистым методом Ньютона) до 316 для f_1 и 389 для f_2 ($t = 0{,}05$), поскольку демпфированный шаг на каждой итерации сокращает расстояние до минимума лишь на долю t, порождая геометрическую сходимость с коэффициентом $(1 - t)$. При этом итоговая точность остаётся высокой при любом t благодаря критерию остановки по норме градиента — демпфирование платит только скоростью, а не точностью.

Таким образом, для чисто квадратичных целевых функций полный (недемпфированный) метод Ньютона является наиболее эффективным: он находит точный минимум за одну итерацию независимо от обусловленности задачи, в то время как демпфирование шага в методе Ньютона-Рафсона целесообразно в первую очередь для функций, на которых квадратичная модель

Ньютона может оказаться неточной или привести к расходимости — для таких функций коэффициент $t < 1$ обеспечивает устойчивость ценой замедления сходимости, что на рассмотренных квадратичных примерах проявляется исключительно как дополнительные вычислительные затраты.